

Programming Assignment 1

Empirical Analysis of the Knuth-Morris-Pratt String Matching Algorithm

Track A: Implementation and Empirical Study

Student: John Zhang

Student ID: 25492238

Course: 41052 Advanced Algorithms

Submission Date: 27/09/2026

Table of Contents

1.0 Project Topic	2
2.0 Project Summary	2
3.0 What I Built	2
4.0 Track A: Empirical Study	4
4.1 Hypotheses	4
4.2 Experimental Method	4
4.3 Experiment 1: Random Input Results	5
4.4 Experiment 2: Adversarial Input	6
4.5 Experiment 3: Increasing Pattern Length	7
4.6 Overall Analysis	9
5.0 What I Learnt	10
6.0 AI Use	11

1.0 Project Topic

The KMP Failure Function

2.0 Project Summary

This project implements the Knuth-Morris-Pratt string matching algorithm and evaluates its practical performance against a naive substring-search baseline. The study investigates how the algorithms behave across different input sizes and string patterns, including random, repetitive and adversarial cases.

3.0 What I Built

For this project, I implemented the Knuth-Morris-Pratt (KMP) string matching algorithm in C++ and compared it against a naive substring-search baseline with the assistance of ChatGPT. KMP and the naive substring search algorithms take a string and compare it with a longer string and search for the target string within the longer string. Its practical applications can include; text editors (Find and Replace), DNA sequencing (finding specific nucleotide patterns (A, C, G, T) inside genomes) and more.

The goal of both algorithms is to find every occurrence of a pattern inside a larger text. The naive algorithm checks the pattern at every possible starting position and compares characters from left to right until either a mismatch occurs or the entire pattern matches, AKA the brute force method. This approach is simple, but worst case it can repeatedly compare the same characters, giving it a time complexity of $O(nm)$, where n is the text length and m is the pattern length.

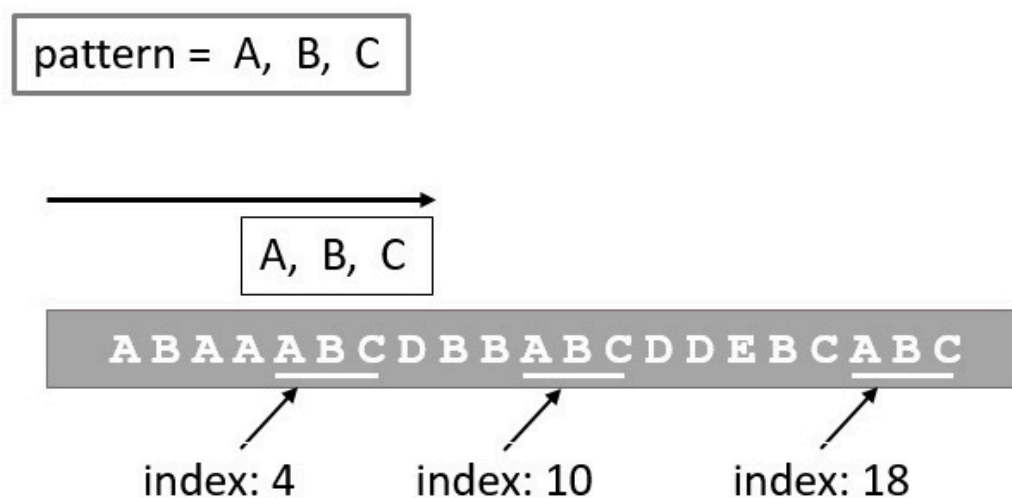


Figure 1 - Naive Pattern Searching Algorithm

The KMP implementation improves on this by preprocessing the pattern into an LPS table, where LPS stands for “Longest Proper Prefix which is also a Suffix”. The LPS table stores information about repeated structure inside the pattern. When a mismatch occurs during the search, KMP uses this table to determine how much of the previous match can still be reused instead of restarting from the beginning of the pattern.

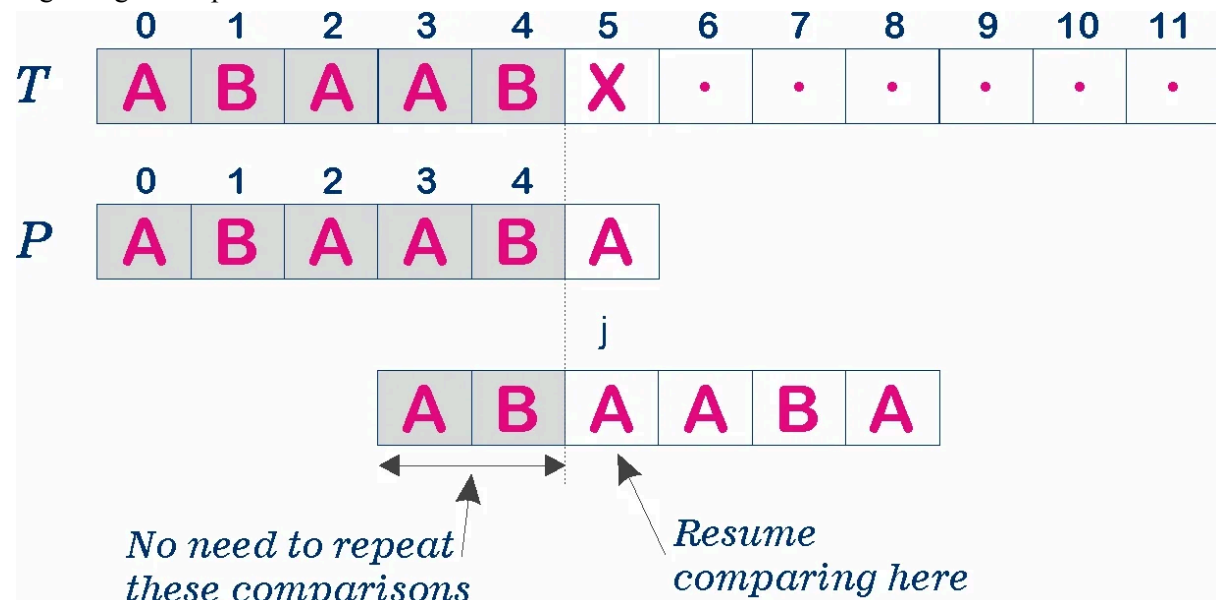


Figure 2 - KMP Pattern Matching image

The implementation is divided into separate source files. The naive search and KMP search are implemented independently so that they can be compared directly. The KMP code is split into two main parts: constructing the LPS table and then using it during the search. Both algorithms return all matching starting indices, including overlapping matches.

I also implemented instrumented versions of both algorithms. These versions perform the same searches but additionally count how many character comparisons are made, allowing the study to compare not only runtime but also the amount of algorithmic work performed independently to speed. To check correctness, I created a test suite with the assistance of ChatGPT containing basic examples, overlapping matches, absent patterns, patterns longer than the text, exact matches and empty string cases. I also added differential testing where KMP and naive search are run on many randomly generated text pattern pairs and their outputs are compared. This helped confirm that both implementations were working before benchmarking.

The implemented benchmark harness generates controlled test inputs, repeatedly time both algorithms, record median runtimes and character comparison counts, verify correctness and export the results to CSV files for analysis and plotting.

4.0 Track A: Empirical Study

The empirical study compared the practical performance of the Knuth-Morris-Pratt algorithm against the naive substring search algorithm. The main research question was:

To what extent does KMP outperform naive substring search in practice and how is this affected by input structure, text length and pattern length?

The study measured both runtime and the number of character comparisons performed by each algorithm. Runtime gives a practical measure of performance on the test machine, while comparison counts provide a more machine independent indication of the amount of algorithmic work being performed.

4.1 Hypotheses

Before performing the experiments, I expected KMP to perform better than naive search as the input became larger because KMP has a worst case time complexity of $O(n+m)$, compared with $O(nm)$ for naive substring search.

However, I also expected the performance difference to depend on the structure of the input. On random strings, mismatches are likely to happen after only a small number of character comparisons. This means that the naive algorithm may often stop comparisons early and therefore may be able to compete despite its theoretical worst case complexity.

For adversarial inputs, I expected KMP to perform much better, as the inputs were deliberately constructed so that the naive algorithm repeatedly matches most of the pattern before failing near the end and restarting. KMP should be able to use its LPS table to avoid repeating as many of these comparisons.

My final hypothesis was that increasing the pattern length under adversarial conditions would have a large effect on naive search but a much smaller effect on KMP.

4.2 Experimental Method

The algorithms were implemented in C++17 and compiled using the Microsoft Visual C++ compiler. Runtime was measured using `std::chrono::steady_clock`. Each search was run 15 times on the same input and the median runtime was recorded. Using the median reduced the effect of occasional timing spikes caused by operating system scheduling or other background processes.

The order in which the two algorithms were measured was alternated between repetitions. This was done to reduce the possibility that one algorithm consistently benefited from running first or second.

Correctness was checked before timing each input by confirming that KMP and naive search returned the same set of matching indices.

Instrumented versions of both algorithms were also implemented under the suggestion of ChatGPT to count character comparisons. Random inputs were generated using a fixed seed and the alphabet; a, b, c and d.

4.3 Experiment 1: Random Input Results

The first experiment investigated performance on randomly generated strings.

The pattern length was fixed at 20 characters, while the text length was increased through:

- 1,000, 5,000, 10,000, 50,000, 100,000 and 500,000 characters.

For each text length, 10 independently generated text pattern pairs were tested. Each individual input was timed 15 times and the median runtime was recorded.

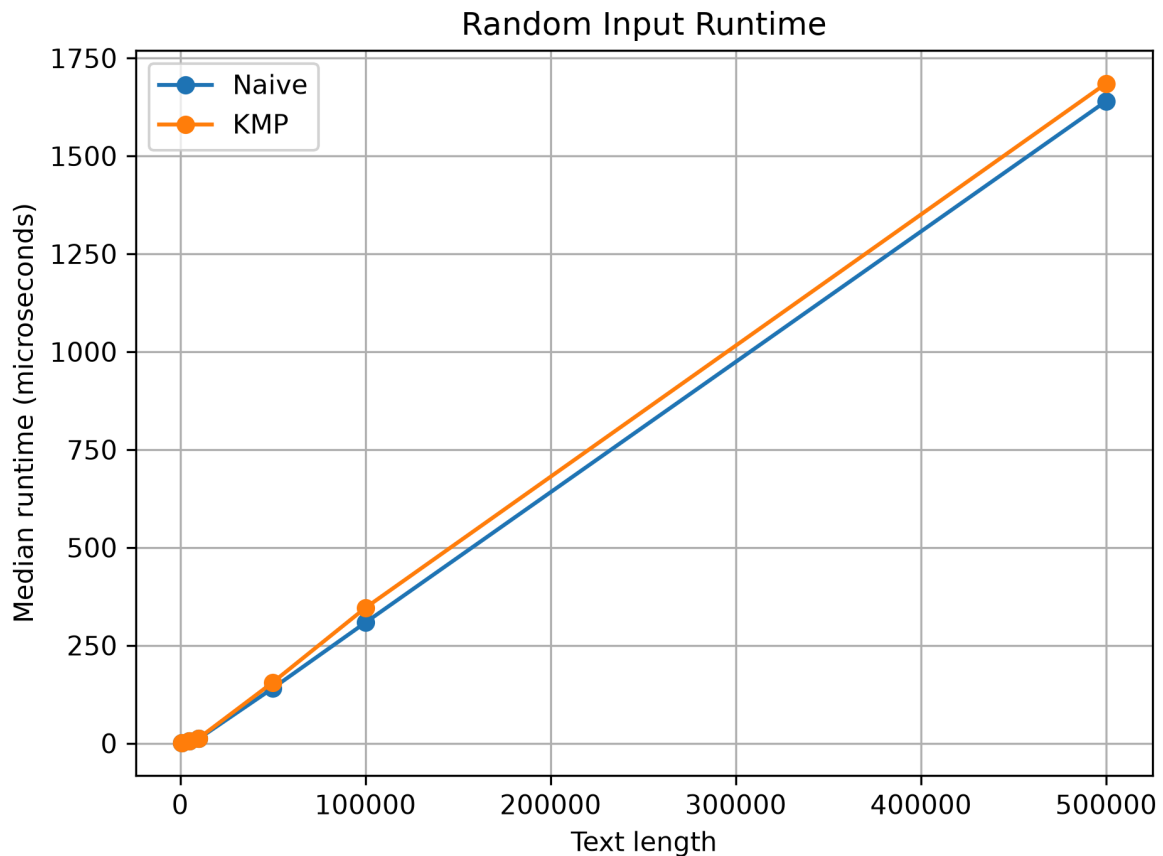


Figure 3 - Random Input Runtime

The results showed that both algorithms scaled approximately linearly as the text became longer. However, the naive implementation was generally slightly faster than KMP on random inputs. This is because random strings often mismatch near the start of the pattern, so the naive algorithm avoids doing the full (m) comparisons. KMP still has preprocessing and extra fallback logic.

4.4 Experiment 2: Adversarial Input

The second experiment was designed to create a difficult case for naive substring search. The text consisted entirely of the character a, while the pattern consisted of 19 a characters followed by a final b.

For example:

a a a a a a a a a a a a a a a a a b

At each possible starting position, the naive search matches almost the entire pattern before failing on the final character. It then moves to the next position and repeats most of the same comparisons. The pattern length remained fixed at 20, while the text length was increased from 1,000 to 500,000 characters.

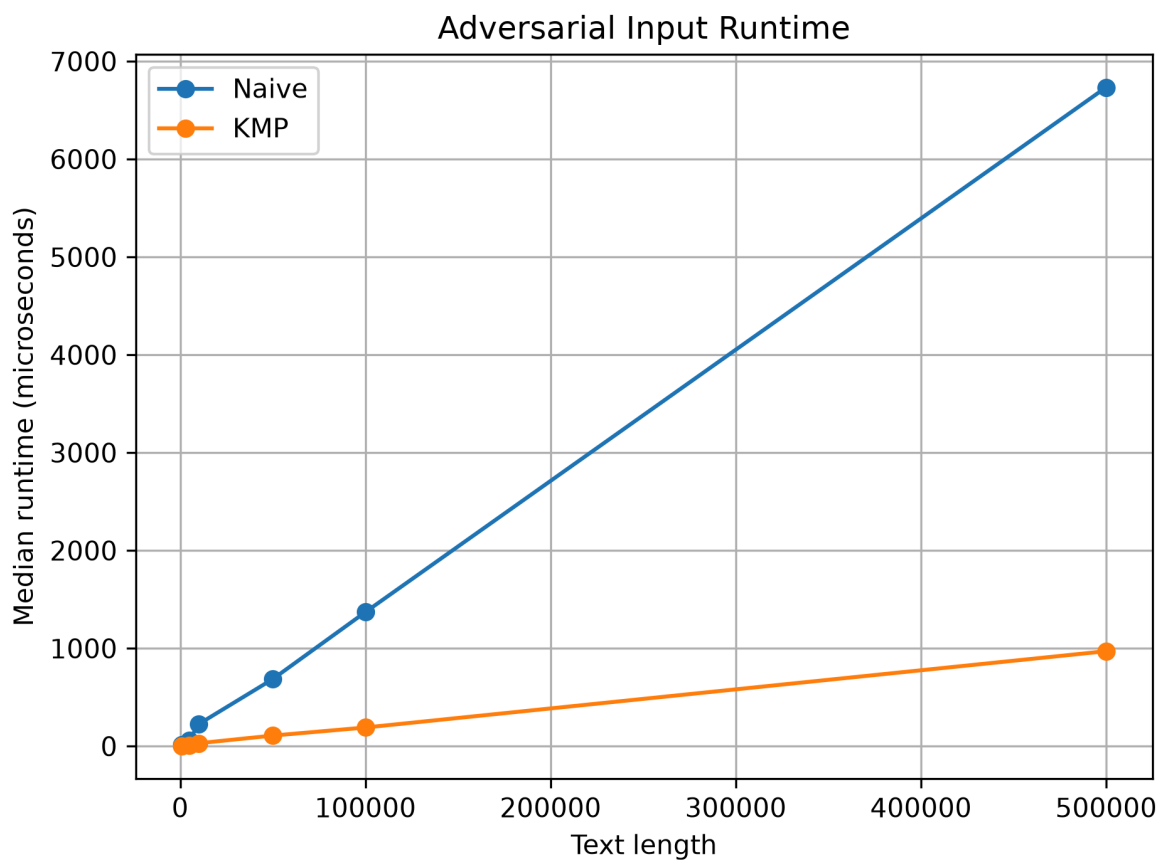


Figure 4 - Adversarial Input Runtime

The results showed a much larger difference between the algorithms than the random input experiment. At a text length of 500,000 characters, the naive algorithm took approximately 6,734 microseconds while KMP took approximately 971 microseconds. The comparison counts help explain this result. At the same input size, naive search performed: 9,999,620 character comparisons. While KMP performed: 999,981 character comparisons.

The naive algorithm therefore performed roughly ten times as many text pattern comparisons.

KMP avoided this repeated work by utilising the LPS table. After a mismatch against the final b, KMP did not discard the entire previous match. Instead, it reused the information that a long suffix of the matched a characters was also a valid prefix of the pattern.

This experiment demonstrates the type of repetitive input for which KMP's preprocessing becomes valuable.

4.5 Experiment 3: Increasing Pattern Length

The third experiment investigated what happens when the pattern length increases while the text length remains fixed.

The text consisted of 100,000 a characters.

The pattern lengths tested were: 10, 20, 40, 80, 160, 320, 640.

Each pattern contained a for every character except the final character, which was b.

For example, a pattern of length 10 had the form:

a a a a a a a a b

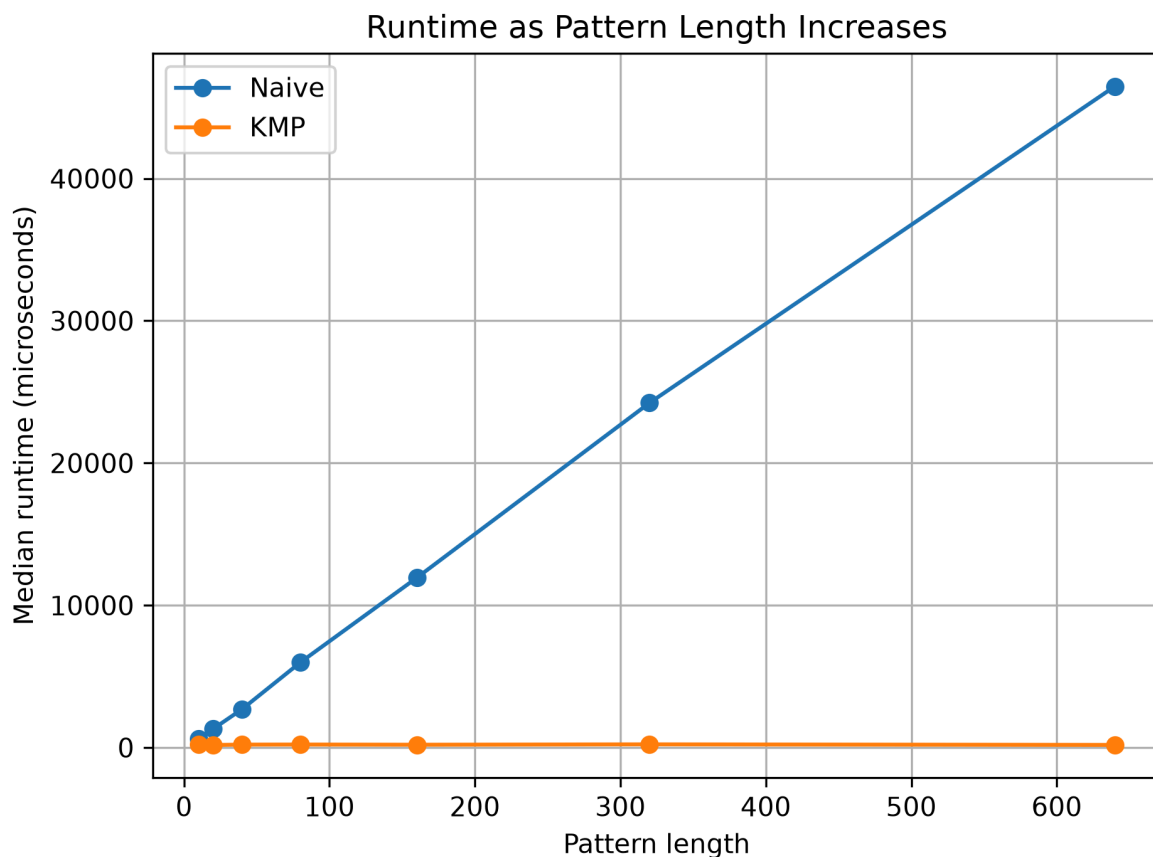


Figure 5 - Runtime as Pattern Length Increases

This experiment produced the largest performance difference. As the pattern length increased, the runtime of naive search increased rapidly. In contrast, KMP remained at approximately the same runtime.

At a pattern length of 640, the naive algorithm took approximately 46,482 microseconds, while KMP took approximately 188 microseconds. A difference of roughly 248 times in runtime.

The character comparison results show why this occurred.

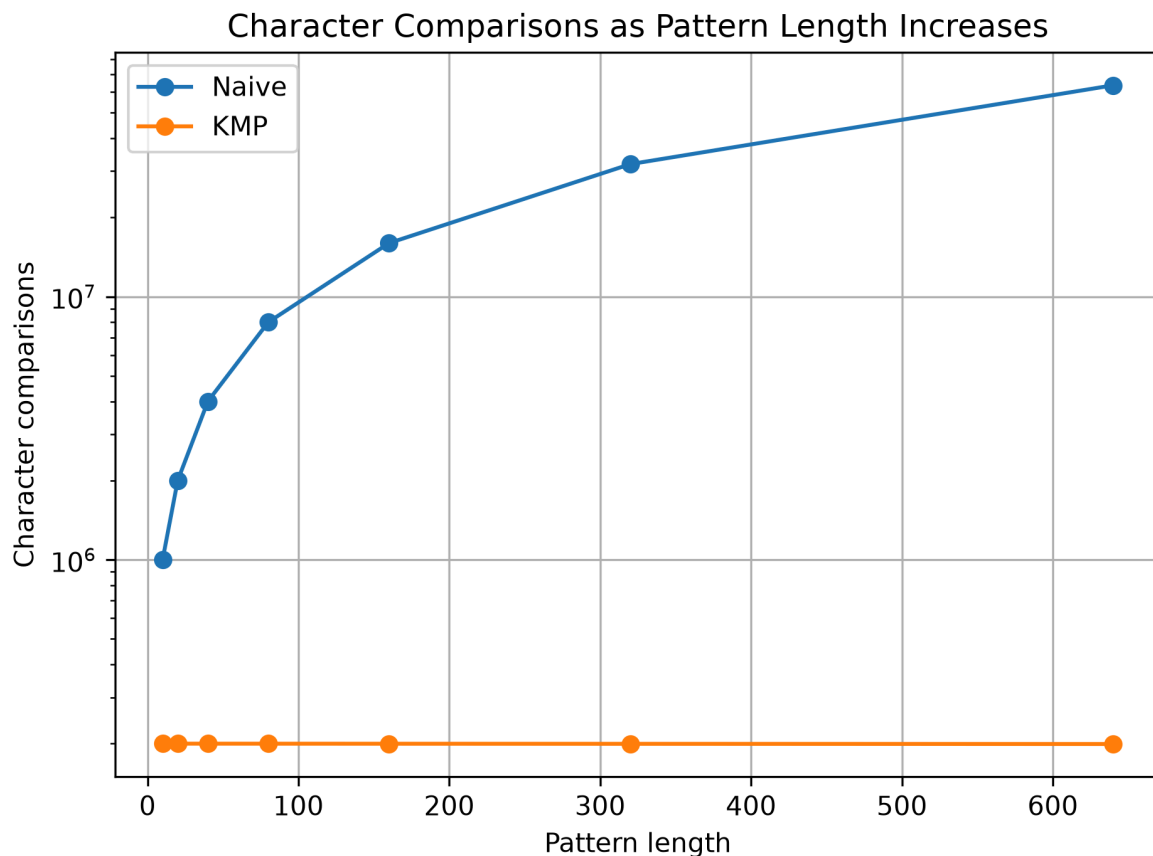


Figure 6 - Character Comparisons as Pattern Length Increases

At pattern length 640, naive search performed: 63,591,040 comparisons.
While KMP performed: 199,361 comparisons.

The naive algorithm's comparison count grew dramatically as the pattern became longer because it repeatedly matched nearly the entire pattern at each possible starting position.

KMP's search phase comparison count remained close to twice the text length. The LPS table allowed the algorithm to reuse the long matched prefix after each mismatch instead of starting again from the beginning.

This experiment showed the clearest difference between the algorithms. As pattern length increased, naive search became much more expensive, while KMP remained almost constant because it avoided repeating previously completed comparisons.

4.6 Overall Analysis

The results showed that the practical advantage of KMP depends strongly on the structure of the input.

On random strings, naive substring search was often slightly faster because mismatches occurred quickly and relatively little repeated work was performed. In these cases, KMP's preprocessing and additional control logic created overhead without providing a large benefit.

On adversarial and highly repetitive inputs, the behaviour was very different. Naive search repeatedly compared long sections of the pattern, while KMP used the LPS table to avoid performing the same comparisons again.

The pattern length experiment showed the strongest difference. Increasing the pattern length caused the cost of naive search to rise substantially, while KMP remained almost constant because the text length was fixed.

Overall the results were consistent with the theoretical complexity analysis, but they also showed an important practical distinction: KMP's stronger worst case complexity does not make it universally faster. Its advantage becomes most visible when the input contains repeated structure that causes naive search to redo large amounts of work.

5.0 What I Learnt

To begin with, when selecting the topic for this assignment I selected a random topic not knowing anything about the Knuth-Morris-Pratt string matching algorithm or the naive substring algorithm either. I had to first understand what each of them were and what their uses were. So to begin with I learnt about what the string algorithms were and what the differences between them were.

One of the main things I learnt from this project was that if something theoretically will be faster, it does not automatically mean it will be faster in practice. Before benchmarking, I had expected KMP to outperform naive substring search on most inputs due to the worst case complexities of both.

The random input experiments surprised me because naive search was often slightly faster than KMP. I initially assumed this meant something was incorrect with my implementation, but the comparison counts helped explain the results of why. On random strings, mismatches usually happen near the beginning of the pattern, so naive search often does very little work before moving on. KMP still has the overhead logic of building the LPS table and using more complex fallback logic.

I also developed a stronger understanding of what the LPS table is and why it is important. After understanding it I first understood it mainly as a table used to “skip ahead”, but now I understand that its purpose is to preserve information about a prefix of the pattern that is already known to match after a mismatch occurs.

The adversarial experiments were the clearest demonstration of this. When the text contained only a characters and the pattern ended in b, naive search repeatedly compared almost the entire pattern before failing. KMP was able to reuse previous work, which caused a large difference in both runtime and character comparisons.

I also learnt that benchmarking algorithms properly requires more than timing them once. Using many forms of measurements, taking the median, using multiple random inputs and character comparison counts made the end results much more reliable, understandable and easier to explain.

6.0 AI Use

I used ChatGPT throughout the project as a programming and explanation assistant. I used it for understanding KMP, naive substring search algorithm, LPS table, assisting with implementation and providing the idea for testing, benchmarking and plotting the code onto the graphs.

ChatGPT was especially useful when I needed explanations or clarification on how KMP or something specific worked or meant. It also helped me create the naive search baseline, a variety of tests, instrumented versions of the algorithms and the benchmark experiments comparing random and adversarial inputs.

I did not take the AI output without checking it. I fact checked when asking for clarification and tested each stage of the implementation and compared KMP against the naive baseline on both written and randomly generated inputs.

The two cases where the AI output needed correction or improvement were the initial benchmarking approach. The original version only generated one random input for each text size and then repeated the timing on that same input. This measured timing noise, but did not capture variation between random strings. I later flagged this and changed the benchmark so that 10 independently generated random text pattern pairs were tested for each size.

Another example was the initial character comparison graph. ChatGPT suggested plotting both algorithms on a normal linear y-axis. Because the naive comparison counts became tens of millions while KMP remained around 200,000, the KMP line was almost flattened against zero and was difficult to interpret. I changed the plot to use a logarithmic y-axis, which made both trends visible.

Now after completing this study on the Knuth-Morris-Pratt string matching algorithm, I understand the main algorithmic ideas, including the role of the LPS table, the difference between naive and KMP search, why adversarial inputs affect the two algorithms differently, which scenarios would be best/better suited for each search and how the benchmark results relate to the theoretical time complexities. I relied more heavily on AI for some implementation as well as testing and benchmarking ideas, however code was reviewed for outputs.